Part-1: Representation of GPIO subsystem in the kernel

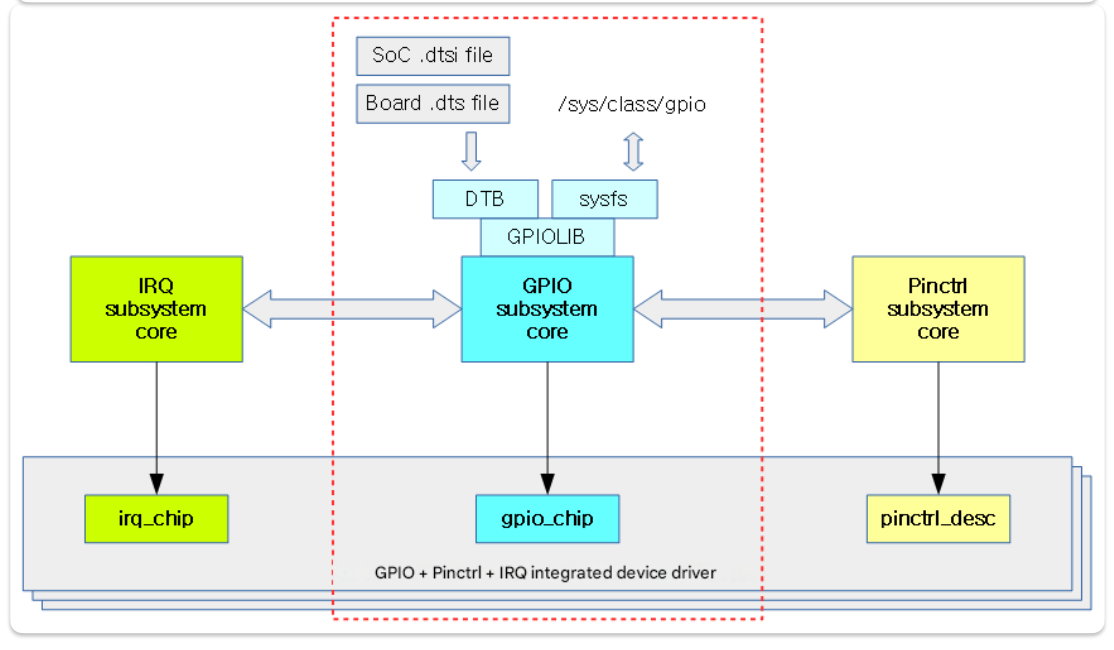
GPIO subsystem

The GPIO subsystem provides frequently used core parts, excluding implementation parts that differ for each GPIO Controller chip. The core provides the DTB parsing part, API, and sysctrl interface connected to user space.

Other subsystems that work with the GPIO subsystem are:

- Pin Control Subsystem
- Interrupt Subsystem with IRQ Domain

The following figure shows the structure of the GPIO subsystem.


SoC .dtsi file
Board .dts file
/sys/class/gpio
DTB
sysfs
GPIOLIB
IRQ subsystem core
GPIO subsystem core
Pinctrl subsystem core
irq_chip
gpio_chip
pinctrl_desc
GPIO + Pinctrl + IRQ integrated device driver

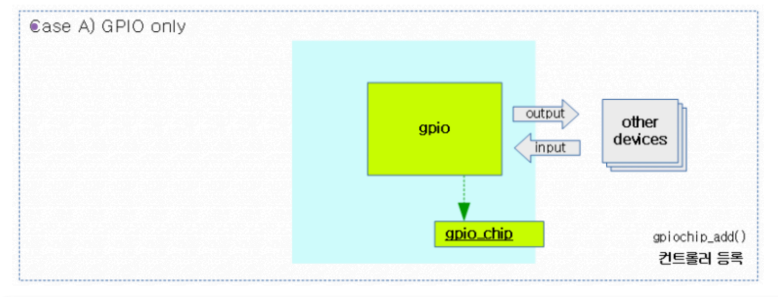
Device Driver Implementation Model

There are three main ways to implement a device driver for a GPIO Controller. The recent trend is to focus on implementing the Pin Controller device driver rather than the GPIO device driver. Therefore, a specific gpio device driver code can be located in the drivers/pinctrl directory as well as the drivers/gpio directory of the kernel directory.

- GPIO only
- GPIO + IRQ
- Pinctrl + GPIO + [IRQ]

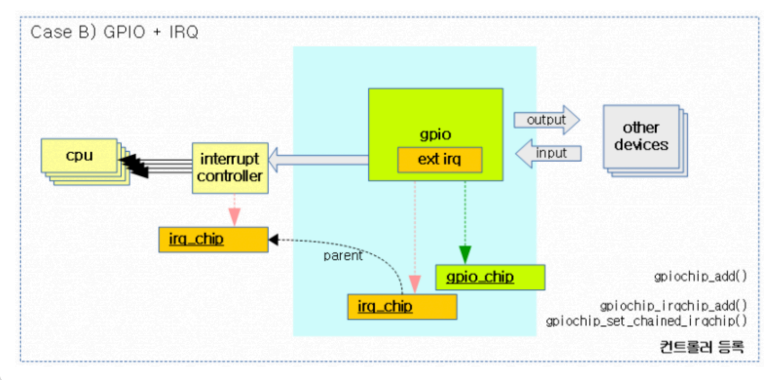
The following figure is an example of a driver for a GPIO device that can only control simple input/output.

- The original role of GPIO is to set the pins to input or output mode and only implement reading or outputting the values.


Case A) GPIO only
gpio
output
input
other devices
gpio_chip
gpiochip_add()
컨트롤러 등록

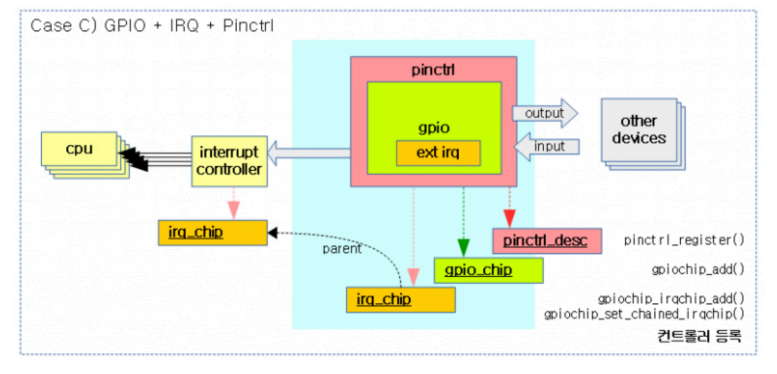
The following figure is an example of a driver for a GPIO device that can handle external interrupts.

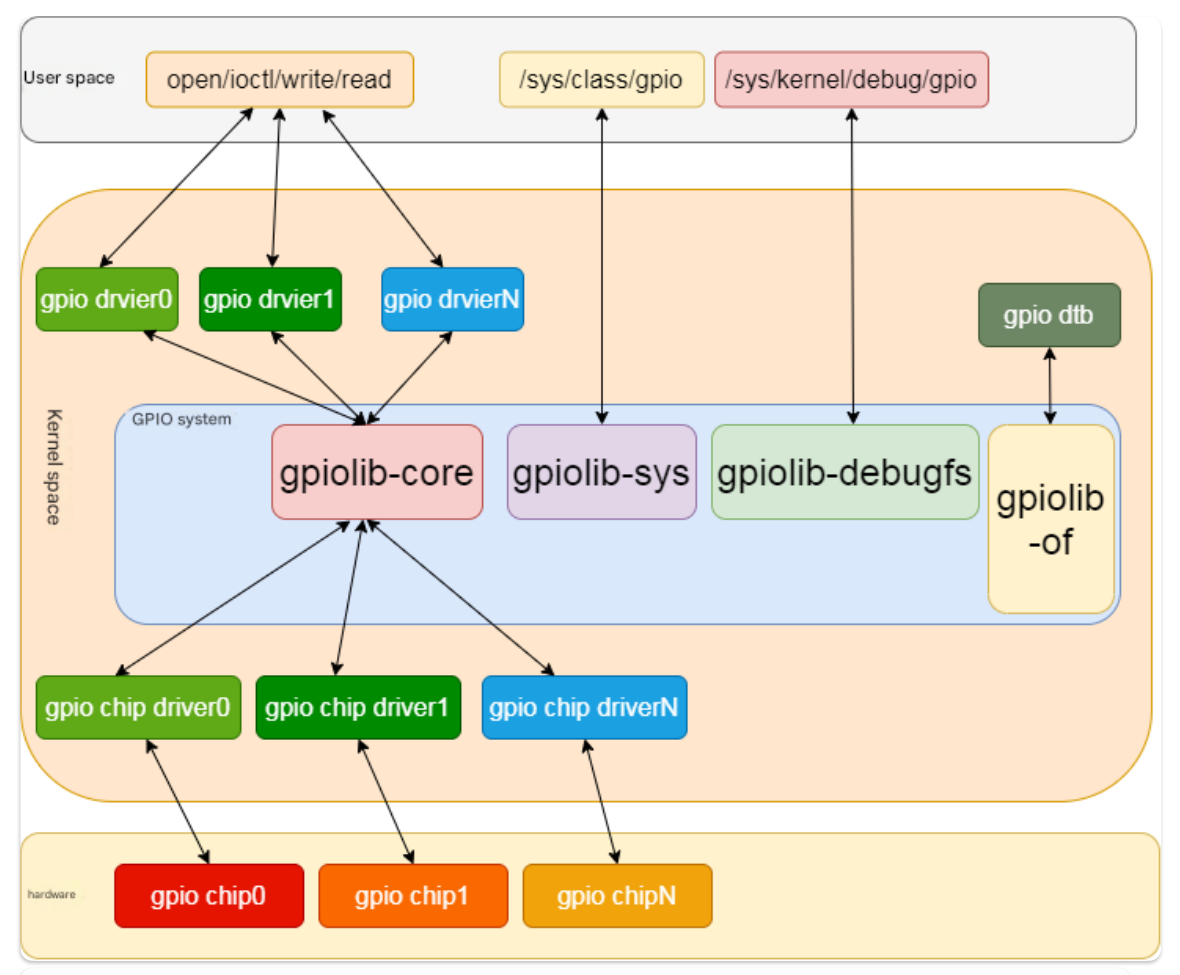
- If the GPIO Controller includes a small IRQ Controller to handle external interrupts, you should implement irq_chip and connect it as a child of the true interrupt controller built into the SoC. There are two ways to configure it, and the main related functions that you should use are as follows.
  - chained irq:
    - Handler operation in irq context (if gpio is inside SoC)
    - gpiochip_irqchip_add()
    - gpiochip_set_chained_irqchip()
  - nested irq:
    - Handler operation in process context (used when i2c requires sleep processing when gpio is attached after i2c)
    - request_threaded_irq()
    - gpiochip_irqchip_add_nested()
    - gpiochip_set_nested_irqchip()

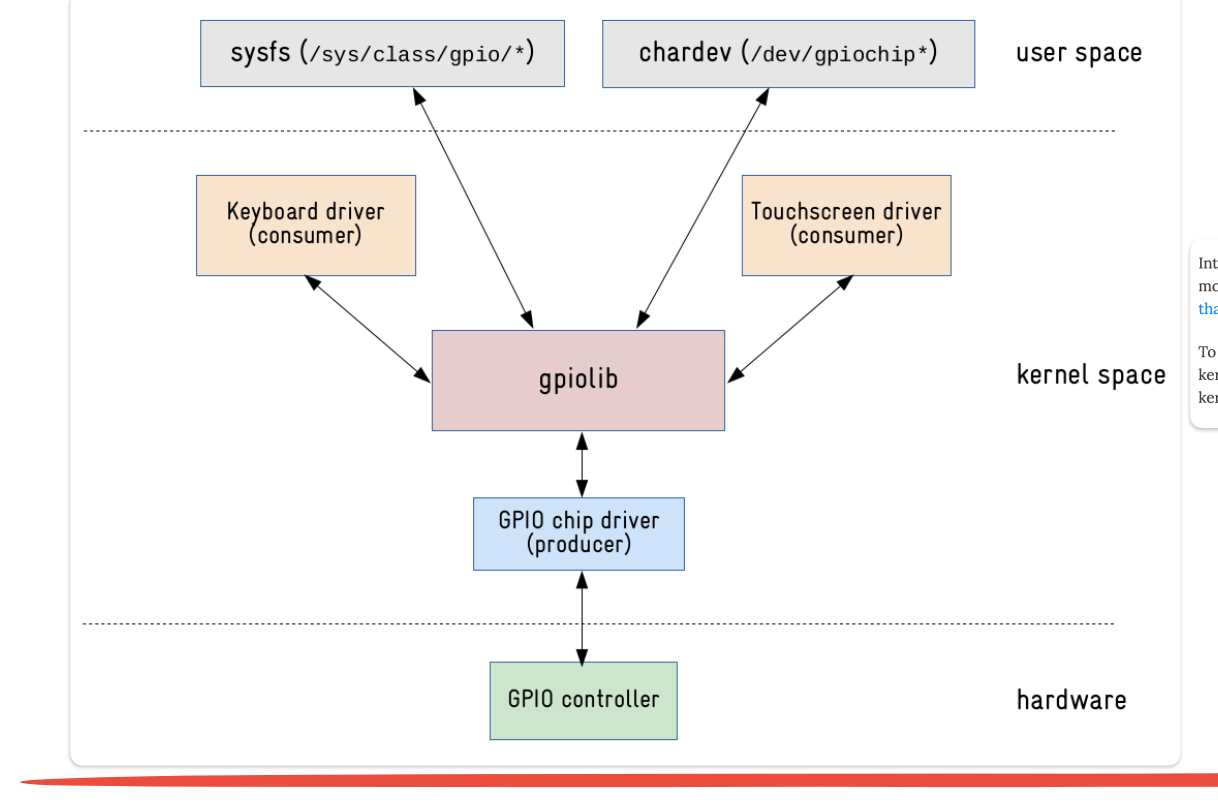

Case B) GPIO + IRQ
cpu
interrupt controller
gpio
ext irq
output
input
other devices
irq_chip
parent
gpio_chip
irq_chip
gpiochip_add()
gpiochip_irqchip_add()
gpiochip_set_chained_irqchip()
컨트롤러 등록

The following figure shows how to additionally interface with a pin controller.

- Depending on the implementation model, there are drivers that implement gpio from the pinctrl driver, and there are also drivers that are implemented separately.
- You can associate a pin controll group with the "gpio-ranges" property on the device tree node that defines the gpio controller.
  - It is also possible to interact with pinctrl and the gpio subsystem in real time via APIs such as pinctrl_add_gpio_range().


Case C) GPIO + IRQ + Pinctrl
pinctrl
gpio
ext irq
output
input
other devices
cpu
interrupt controller
irq_chip
parent
pinctrl_desc
gpio_chip
irq_chip
pinctrl_register()
gpiochip_add()
gpiochip_irqchip_add()
gpiochip_set_chained_irqchip()
컨트롤러 등록


User space
open/ioctl/write/read
/sys/class/gpio
/sys/kernel/debug/gpio
Kernel space
gpio drvier0
gpio drvier1
gpio drvierN
gpio dtb
GPIO system
gpiolib-core
gpiolib-sys
gpiolib-debugfs
gpiolib-of
gpio chip driver0
gpio chip driver1
gpio chip driverN
hardware
gpio chip0
gpio chip1
gpio chipN


sysfs (/sys/class/gpio/*)
chardev (/dev/gpiochip*)
user space
Keyboard driver (consumer)
Touchscreen driver (consumer)
gpiolib
kernel space
GPIO chip driver (producer)
GPIO controller
hardware

Internally, the Linux kernel implements the access to GPIOs via a producer/consumer model. There are drivers that produce GPIO lines (GPIO controllers drivers) and drivers that consume GPIO lines (keyboard, touchscreen, sensors, etc).

To manage the GPIO registration and allocation there is a framework inside the Linux kernel called **gpiolib**. This framework provides an API to both device drivers running in kernel space and user space applications.

GPIO Subsystem structures and APIs

GPIO Descriptor Based APIs (GPIOLIB)

The following GPIO descriptor-based APIs are provided (functions and arguments are omitted):

- gpio_to_desc()
- desc_to_gpio()
- gpiod_to_chip()
- gpiod_get_direction()
- gpiod_direction_input()
- gpiod_direction_output_raw()
- gpiod_direction_output()
- gpiod_set_debounce()
- gpiod_is_active_low()
- gpiod_get_raw_value()
- gpiod_get_value()
- gpiod_set_raw_value()
- gpiod_set_value()
- gpiod_set_raw_array_value()
- gpiod_set_array_value()
- gpiod_cansleep()
- gpiod_to_irq()
- gpiod_get_raw_value_cansleep()
- gpiod_get_value_cansleep()
- gpiod_set_raw_value_cansleep()
- gpiod_set_value_cansleep()
- gpiod_set_raw_array_value_cansleep()
- gpiod_set_array_value_cansleep()
- gpiod_add_lookup_table()
- gpiod_remove_lookup_table()
- gpiod_count()
- gpiod_get()
- gpiod_get_optional()
- gpiod_get_index()
- fwnode_get_named_gpiod()
- gpiod_get_index_optional()
- gpiod_put()
- gpiod_get_array()
  - Added in kernel v4.1-rc1.
  - Note: gpiolib: add gpiod_get_array and gpiod_put_array functions
- gpiod_get_array_optional()
- gpiod_put_array()

GPIO Controller APIs (GPIOLIB)

The following APIs related to GPIO controller are provided. (Function types and arguments are omitted.)

- gpiochip_add_data() & devm_gpiochip_add_data()
- gpiochip_get_data()
- gpiochip_remove() & devm_gpiochip_remove()
- gpiochip_find()
- gpiochip_set_chained_irqchip()
- gpiochip_set_nested_irqchip()
- gpiochip_irqchip_add_key()
- gpiochip_generic_request()
- gpiochip_generic_config()
- gpiochip_generic_free()
- gpiochip_is_requested()
- gpiochip_request_own_desc()
- gpiochip_free_own_desc()
- gpiochip_lock_as_irq()
- gpiochip_unlock_as_irq()
- gpiochip_line_is_irq()
- gpiochip_line_is_open_drain()
- gpiochip_line_is_open_source()
- gpiochip_line_is_persistent()

GPIO Descriptor

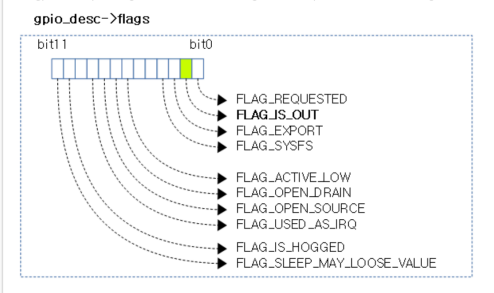
drivers/gpio/gpiolib.h

```c
struct gpio_desc {
        struct gpio_device      *gdev;
        unsigned long           flags;
/* flag symbols are bit numbers */
#define FLAG_REQUESTED  0
#define FLAG_IS_OUT     1
#define FLAG_EXPORT     2       /* protected by sysfs_lock */
#define FLAG_SYSFS      3       /* exported via /sys/class/gpio/control */
#define FLAG_ACTIVE_LOW 6       /* value has active low */
#define FLAG_OPEN_DRAIN 7       /* Gpio is open drain type */
#define FLAG_OPEN_SOURCE 8      /* Gpio is open source type */
#define FLAG_USED_AS_IRQ 9      /* GPIO is connected to an IRQ */
#define FLAG_IS_HOGGED  11      /* GPIO is hogged */
#define FLAG_SLEEP_MAY_LOOSE_VALUE 12   /* GPIO may loose value in sleep */

        /* Connection label */
        const char              *label;
        /* Name of the GPIO */
        const char              *name;
};
```

- flags
  - gpio pin flags
- *label
  - gpio pin label names
  - If the label name is not specified when exposed with the export command in "/sys/class/gpio", the label name "sysfs" is used.
  - If a gpio pin is used for an interrupt and no label name is specified, the label name "interrupt" is used.
  - When requested and used in the device tree, a consumer id name is specified.
    - Example) "<con_id>-gpios = <>"
- *name
  - gpio pin names
  - Specifies when set as a line name in the device tree.
    - Example) gpio-line-names = "MMC-CD", "NC";

The gpio descriptor flag bits, which indicate flags for GPIO pins, have the following characteristics:


gpio_desc->flags
bit11
bit0
FLAG_REQUESTED
FLAG_IS_OUT
FLAG_EXPORT
FLAG_SYSFS
FLAG_ACTIVE_LOW
FLAG_OPEN_DRAIN
FLAG_OPEN_SOURCE
FLAG_USED_AS_IRQ
FLAG_IS_HOGGED
FLAG_SLEEP_MAY_LOOSE_VALUE

- FLAG_REQUESTED
  - Set when a request to use a gpio pin is made by a user (Consumer).
  - -EBUSY error when accessed by other users
- FLAG_IS_OUT
  - Input (0)/Output (1) mode
- FLAG_EXPORT
  - When exposed by the export command in gpiod_export() or "/sys/class/gpio".
- FLAG_SYSFS
  - Set when exposed with the export command in "/sys/class/gpio".
- FLAG_ACTIVE_LOW
  - The gpio state is set in the circuit designed to react active at a low (0) value.
  - The attribute value "X-gpios = <&phandle gpio_num GPIO_ACTIVE_LOW>" is applied in the device tree.
- FLAG_OPEN_DRAIN
  - Set when the gpio internal circuit setting is open drain.
- FLAG_OPEN_SOURCE
  - Set if the gpio internal circuit settings are open source.
- FLAG_USED_AS_IRQ
  - Set when a gpio pin is connected to an irq.
- FLAG_IS_HOGGED
  - Set when the gpio pin is hogged.
- FLAG_SLEEP_MAY_LOOSE_VALUE
  - The gpio pin loses its gpio value when in power-saving sleep.
  - If this bit is set, the gpio controller will reset the gpio values on wakeup.

How a gpio pin gets a descriptor value(i.e, gpio_to_desc()) ?

Convert gpio number to gpio descriptor

gpio_to_desc()

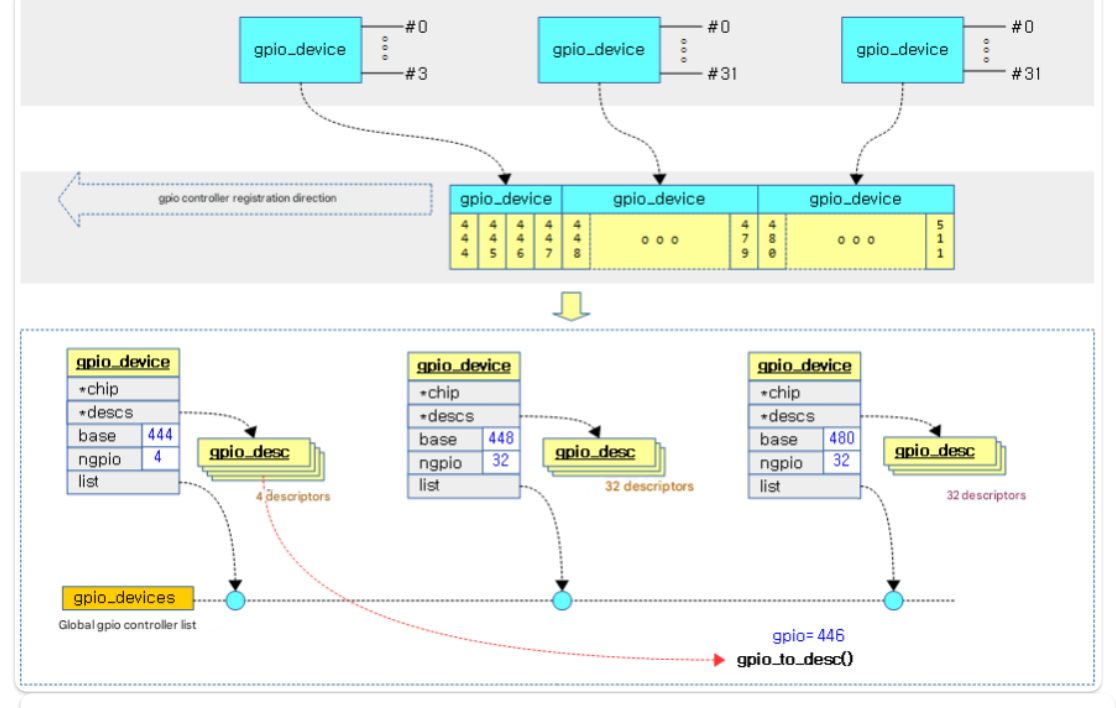
drivers/gpio/gpiolib.c

```c
01 /**
02  * gpio_to_desc - Convert a GPIO number to its descriptor
03  * @gpio: global GPIO number
04  *
05  * Returns:
06  * The GPIO descriptor associated with the given GPIO, or %NULL if no GPIO
07  * with the given number exists in the system.
08  */
09 struct gpio_desc *gpio_to_desc(unsigned gpio)
10 {
11         struct gpio_device *gdev;
12         unsigned long flags;
13 
14         spin_lock_irqsave(&gpio_lock, flags);
15 
16         list_for_each_entry(gdev, &gpio_devices, list) {
17                 if (gdev->base <= gpio &&
18                     gdev->base + gdev->ngpio > gpio) {
19                         spin_unlock_irqrestore(&gpio_lock, flags);
20                         return &gdev->descs[gpio - gdev->base];
21                 }
22         }
23 
24         spin_unlock_irqrestore(&gpio_lock, flags);
25 
26         if (!gpio_is_valid(gpio))
27                 WARN(1, "invalid GPIO %d\n", gpio);
28 
29         return NULL;
30 }
31 EXPORT_SYMBOL_GPL(gpio_to_desc);
```

Converts a gpio number to a gpio descriptor.

- In lines 14-24 of the code, the gpio_lock spinlock is acquired, the global gpio_devices list is traversed, and the descriptor managed by the gpio_device containing the requested gpio number is returned.

The following figure shows finding gpio 446 and returning a descriptor.


gpio_device
#0
#3
gpio_device
#0
#31
gpio_device
#0
#31
gpio controller registration direction
gpio_device
gpio_device
gpio_device
444 445 446 447 448 ... 479 480 ... 511
gpio_device
*chip
*descs
base 444
ngpio 4
list
gpio_desc
4 descriptors
gpio_device
*chip
*descs
base 448
ngpio 32
list
gpio_desc
32 descriptors
gpio_device
*chip
*descs
base 480
ngpio 32
list
gpio_desc
32 descriptors
gpio_devices
Global gpio controller list
gpio= 446
gpio_to_desc()

desc_to_gpio()

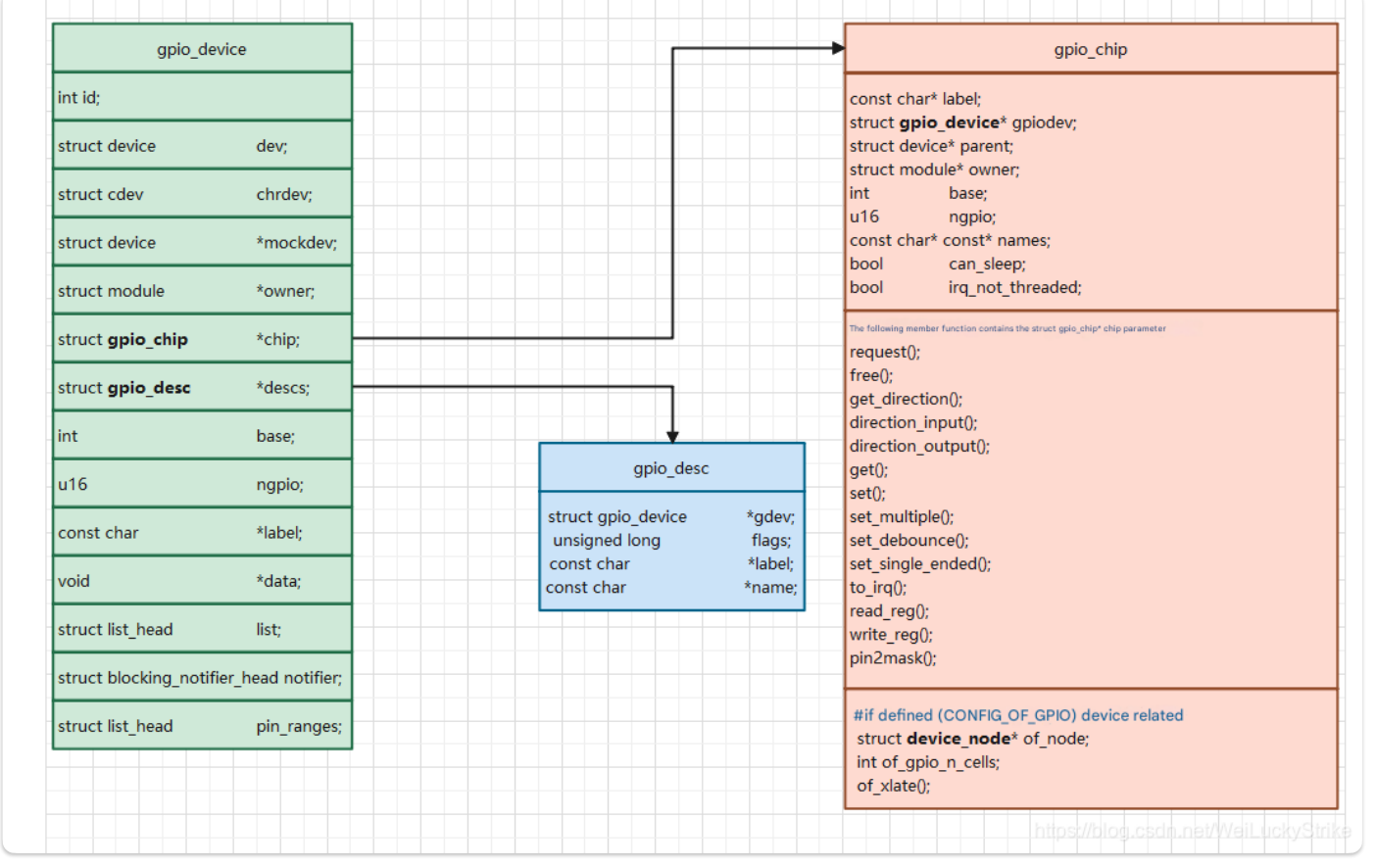
drivers/gpio/gpiolib.c

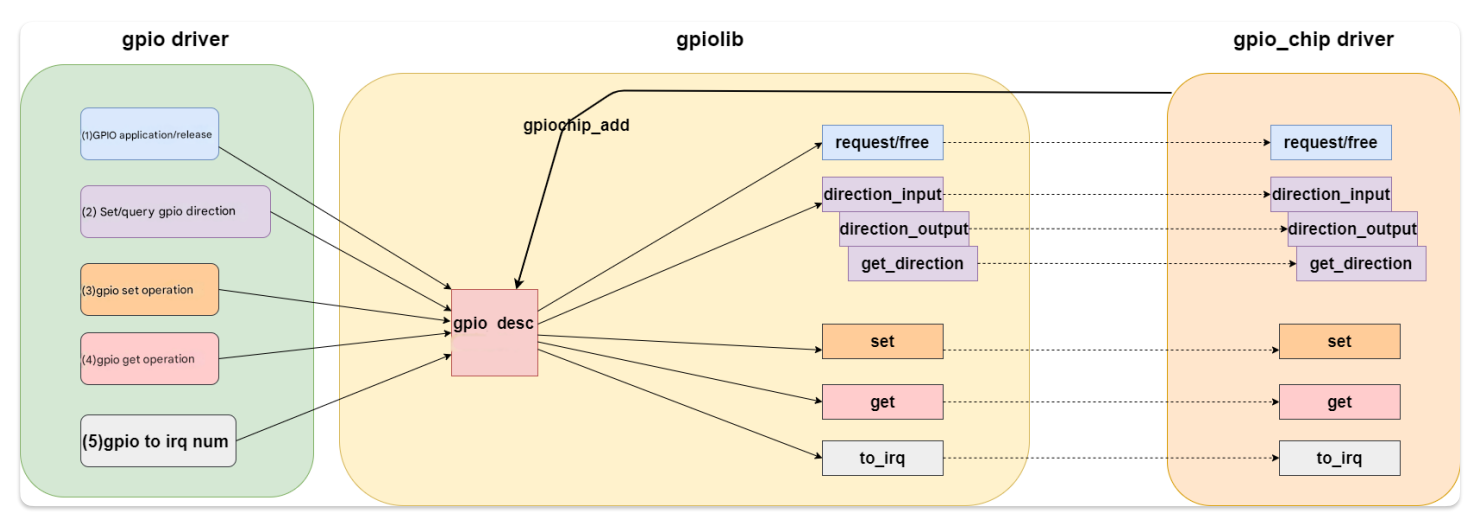
```c
/**
 * desc_to_gpio - convert a GPIO descriptor to the integer namespace
 * @desc: GPIO descriptor
 *
 * This should disappear in the future but is needed since we still
 * use GPIO numbers for error messages and sysfs nodes.
 *
 * Returns:
 * The global GPIO number for the GPIO specified by its descriptor.
 */
int desc_to_gpio(const struct gpio_desc *desc)
{
        return desc->gdev->base + (desc - &desc->gdev->descs[0]);
}
EXPORT_SYMBOL_GPL(desc_to_gpio);
```

Find out by converting it to a gpio number using the gpio descriptor given as an argument.

- The pio descriptor has a gdev pointer member that points to a gpio_device structure. This returns the offset location of the gpio descriptor relative to the gpio starting number (base) in the gpio_device.
  - The gpio descriptor array is a one-dimensional array consisting of the base number and the number of ngpios.

GPIO Structure relationship

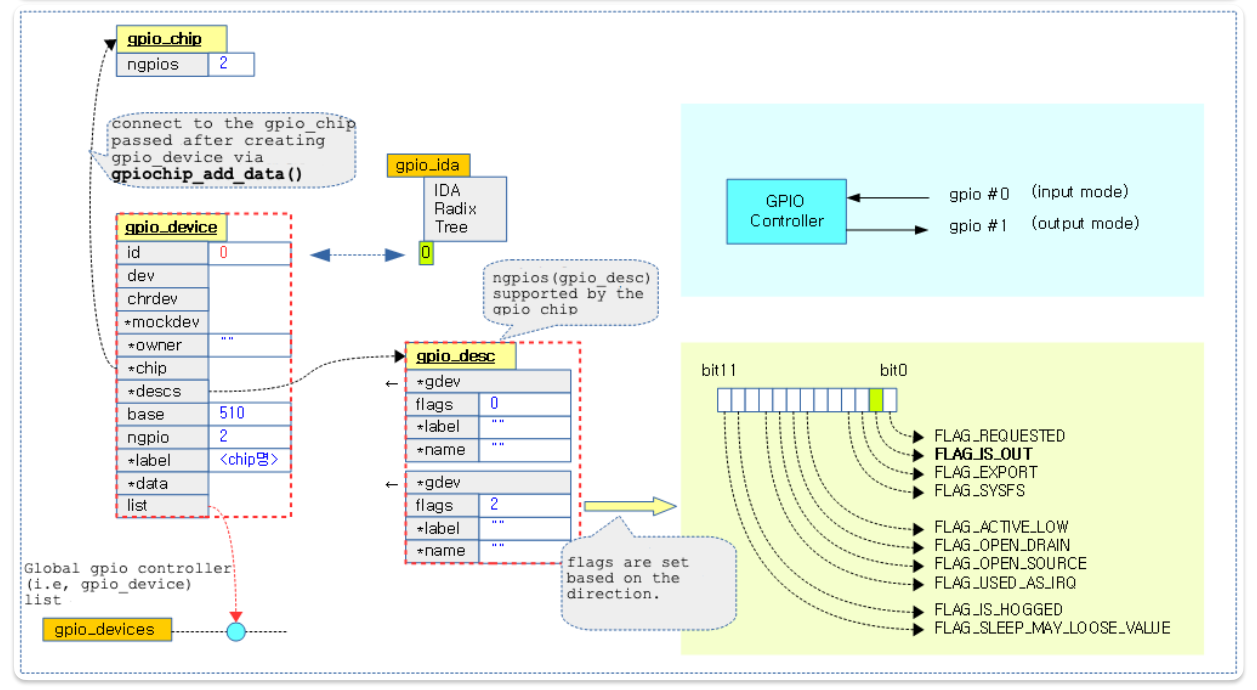

gpio_device
int id;
struct device dev;
struct cdev chrdev;
struct device *mockdev;
struct module *owner;
struct gpio_chip *chip;
struct gpio_desc *descs;
int base;
u16 ngpio;
const char *label;
void *data;
struct list_head list;
struct blocking_notifier_head notifier;
struct list_head pin_ranges;
gpio_desc
struct gpio_device *gdev;
unsigned long flags;
const char *label;
const char *name;
gpio_chip
const char* label;
struct gpio_device* gpiodev;
struct device* parent;
struct module* owner;
int base;
u16 ngpio;
const char* const* names;
bool can_sleep;
bool irq_not_threaded;
The following member function contains the struct gpio_chip* chip parameter
request();
free();
get_direction();
direction_input();
direction_output();
get();
set();
set_multiple();
set_debounce();
set_single_ended();
to_irq();
read_reg();
write_reg();
pin2mask();
#if defined (CONFIG_OF_GPIO) device related
struct device_node* of_node;
int of_gpio_n_cells;
of_xlate();


gpio driver
(1)GPIO application/release
(2) Set/query gpio direction
(3)gpio set operation
(4)gpio get operation
(5)gpio to irq num
gpiolib
gpiochip_add
gpio_desc
request/free
direction_input
direction_output
get_direction
set
get
to_irq
gpio_chip driver
request/free
direction_input
direction_output
get_direction
set
get
to_irq

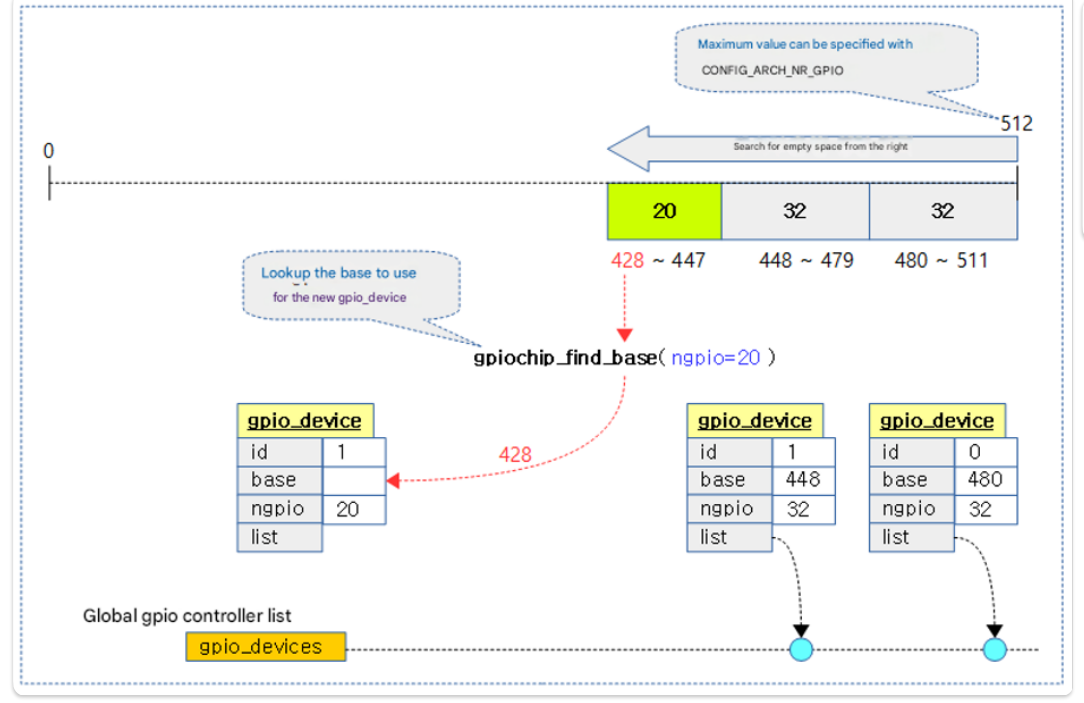
Part - 2: How a gpio_chip is registered and how gpio_device, gpio_chip, gpio_desc structures are associated ?

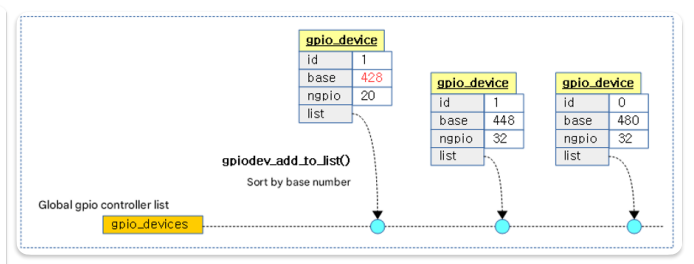
Register GPIO Controller (gpio_chip)

The figure below shows how to register a gpio controller with two gpio pins.

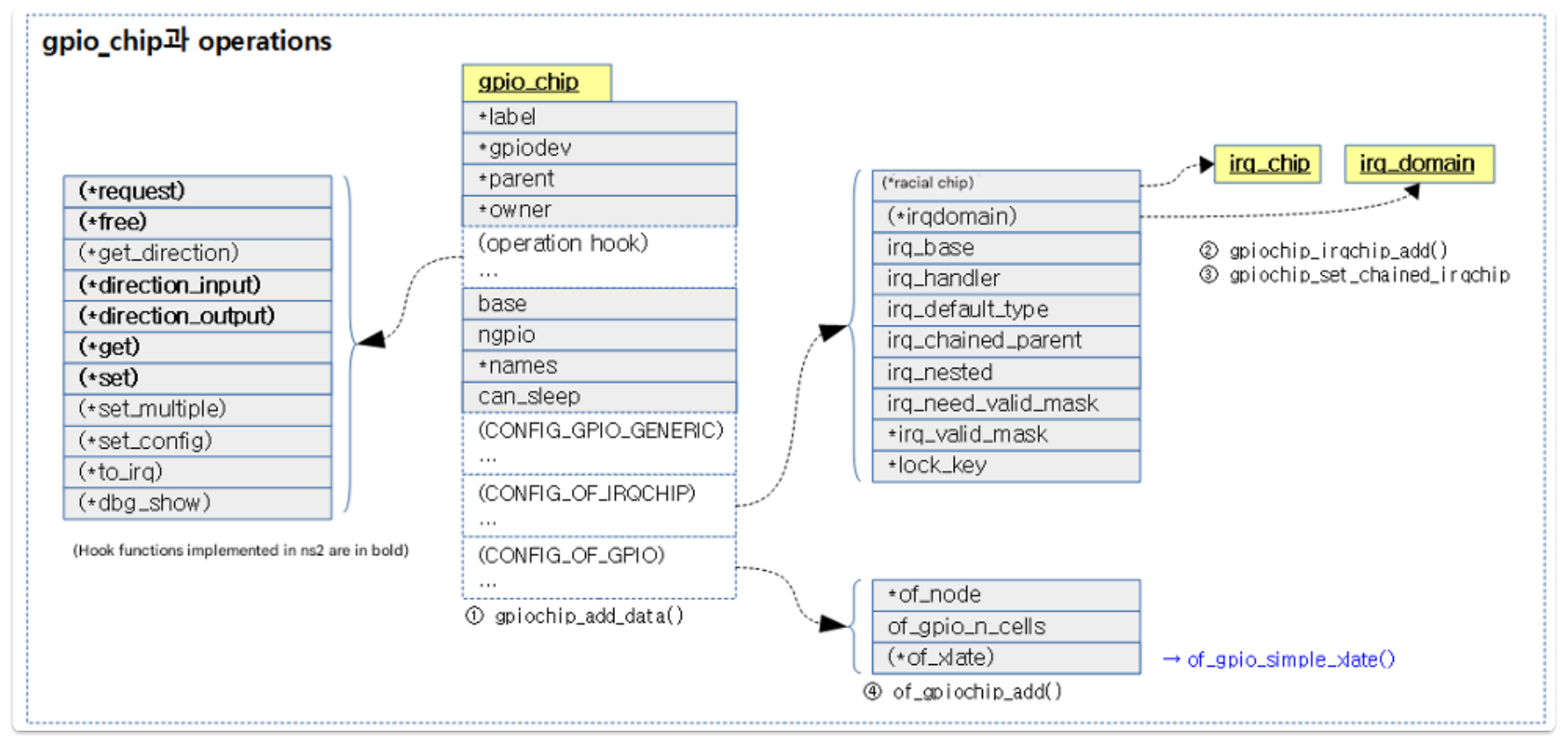

gpio_chip
ngpios 2
connect to the gpio_chip passed after creating gpio device via gpiochip_add_data()
gpio_ida
IDA Radix Tree
0
gpio_device
id 0
dev
chrdev
*mockdev
*owner
*chip
*descs
base 510
ngpio 2
*label <chip명>
*data
list
ngpios(gpio_desc) supported by the gpio chip
gpio_desc
*gdev
flags 0
*label
*name
*gdev
flags 2
*label
*name
flags are set based on the direction.
Global gpio controller (i.e, gpio_device) list
gpio_devices
GPIO Controller
gpio #0 (input mode)
gpio #1 (output mode)
bit11
bit0
FLAG_REQUESTED
FLAG_IS_OUT
FLAG_EXPORT
FLAG_SYSFS
FLAG_ACTIVE_LOW
FLAG_OPEN_DRAIN
FLAG_OPEN_SOURCE
FLAG_USED_AS_IRQ
FLAG_IS_HOGGED
FLAG_SLEEP_MAY_LOOSE_VALUE

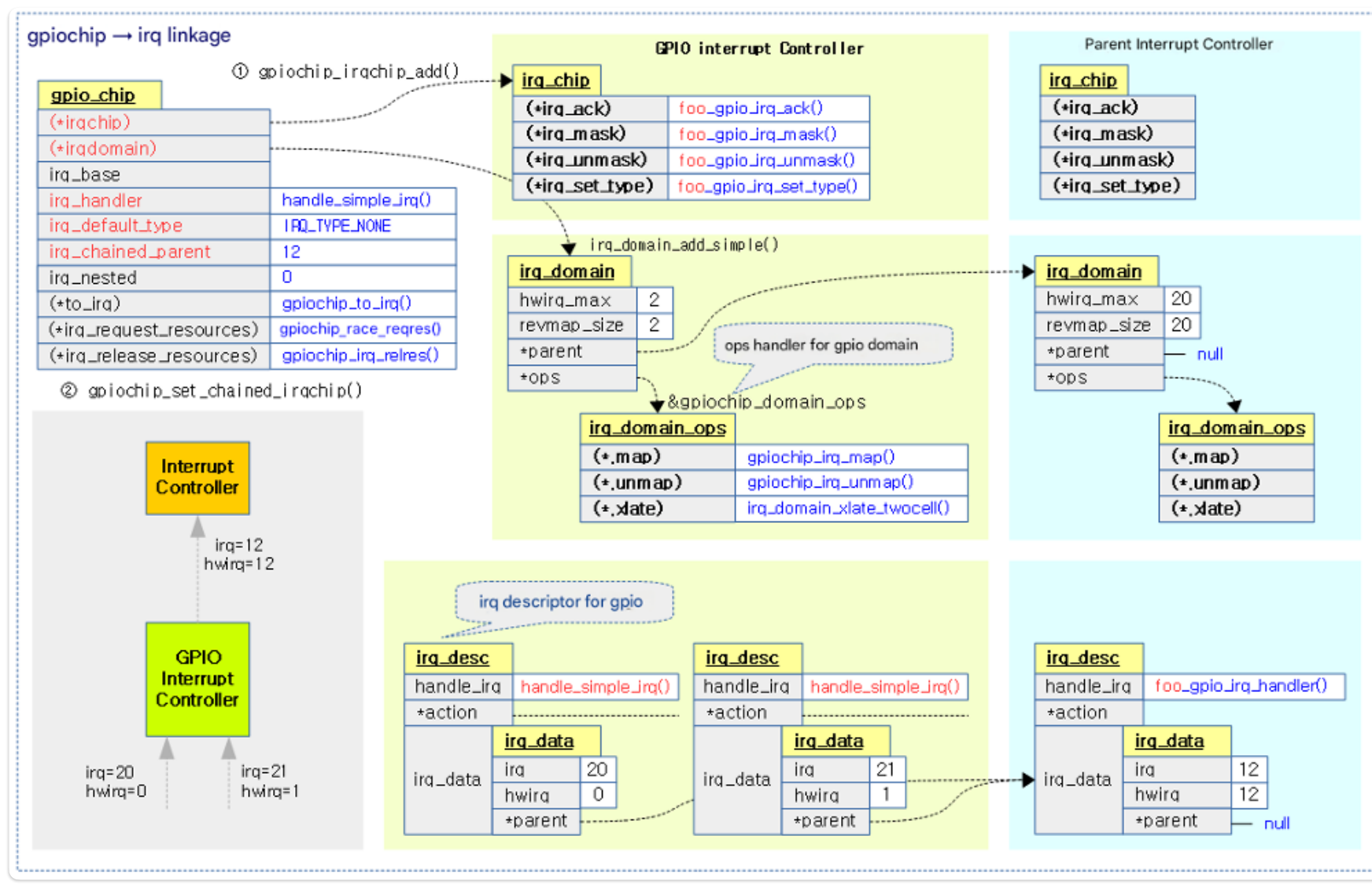
As part of gpiochip_add_data() function how struct gpio_device gets it's base index ?


Maximum value can be specified with CONFIG_ARCH_NR_GPIO
512
0
Search for empty space from the right
20
32
32
428 ~ 447
448 ~ 479
480 ~ 511
Lookup the base to use for the new gpio_device
gpiochip_find_base(ngpio=20)
428
gpio_device
id 1
base
ngpio 20
list
gpio_device
id 1
base 448
ngpio 32
list
gpio_device
id 0
base 480
ngpio 32
list
Global gpio controller list
gpio_devices


gpio_device
gpiodev_add_to_list()
Sort by base number
Global gpio controller list
gpio_devices

Part - 3: How a gpio_chip and irq_chip is registered ?


gpio_chip과 operations
gpio_chip
*label
*gpiodev
*parent
*owner
(operation hook)
...
base
ngpio
*names
can_sleep
(CONFIG_GPIO_GENERIC)
...
(CONFIG_OF_IRQCHIP)
...
(CONFIG_OF_GPIO)
...
① gpiochip_add_data()
(*request)
(*free)
(*get_direction)
(*direction_input)
(*direction_output)
(*get)
(*set)
(*set_multiple)
(*set_config)
(*to_irq)
(*dbg_show)
(Hook functions implemented in ns2 are in bold)
(*racial chip)
(*irqdomain)
irq_base
irq_handler
irq_default_type
irq_chained_parent
irq_nested
irq_need_valid_mask
*irq_valid_mask
*lock_key
irq_chip
irq_domain
② gpiochip_irqchip_add()
③ gpiochip_set_chained_irqchip
*of_node
of_gpio_n_cells
(*of_xlate)
→ of_gpio_simple_xlate()
④ of_gpiochip_add()


gpiochip → irq linkage
① gpiochip_irqchip_add()
gpio_chip
(*irqchip)
(*irqdomain)
irq_base
irq_handler handle_simple_irq()
irq_default_type IRQ_TYPE_NONE
irq_chained_parent 12
irq_nested 0
(*to_irq) gpiochip_to_irq()
(*irq_request_resources) gpiochip_irq_reqres()
(*irq_release_resources) gpiochip_irq_relres()
② gpiochip_set_chained_irqchip()
GPIO interrupt Controller
irq_chip
(*irq_ack) foo_gpio_irq_ack()
(*irq_mask) foo_gpio_irq_mask()
(*irq_unmask) foo_gpio_irq_unmask()
(*irq_set_type) foo_gpio_irq_set_type()
irq_domain_add_simple()
irq_domain
hwirq_max 2
revmap_size 2
*parent
*ops
ops handler for gpio domain
&gpiochip_domain_ops
irq_domain_ops
(*.map) gpiochip_irq_map()
(*.unmap) gpiochip_irq_unmap()
(*.xlate) irq_domain_xlate_twocell()
Parent Interrupt Controller
irq_chip
(*irq_ack)
(*irq_mask)
(*irq_unmask)
(*irq_set_type)
irq_domain
hwirq_max 20
revmap_size 20
*parent null
*ops
irq_domain_ops
(*.map)
(*.unmap)
(*.xlate)
Interrupt Controller
irq=12
hwirq=12
GPIO Interrupt Controller
irq=20
hwirq=0
irq=21
hwirq=1
irq descriptor for gpio
irq_desc
handle_irq handle_simple_irq()
*action
irq_data
irq_data
irq 20
hwirq 0
*parent
irq_desc
handle_irq handle_simple_irq()
*action
irq_data
irq_data
irq 21
hwirq 1
*parent
irq_desc
handle_irq foo_gpio_irq_handler()
*action
irq_data
irq_data
irq 12
hwirq 12
*parent null

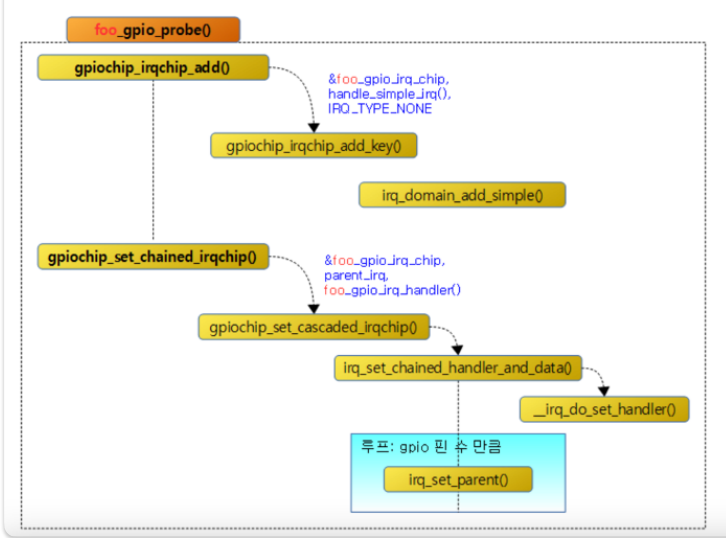
Part - 4: How a gpio_chip and irq_chip is associated and how gpio controllers are treated as interrupt controllers ?

Cascade linkage with upper level interrupt controller

Cascade communication between interrupt controllers can be accomplished using one of two methods:

- chained irq
  - Interrupt handlers run in irq context.
  - Do not use slow APIs when calling sub-device (gpio) registers in interrupt handlers (blocking APIs are prohibited).
  - Used when connecting the gpio connected inside the SoC.
  - You can also use the gpio connected behind the PCI bus (since PCI is faster).
- nested irq
  - The interrupt handler runs in process context, i.e. the RT thread wakes up.
  - It is possible to use slow APIs when calling sub-device (gpio) registers from interrupt handlers.
  - It is mainly used when a device such as GPIO is connected behind an i2c or spi bus.

The following figure shows the function call process for chained irq linkage in the probe function, which is the entry point of the gpio controller device driver.


foo_gpio_probe()
gpiochip_irqchip_add()
&foo_gpio_irq_chip,
handle_simple_irq(),
IRQ_TYPE_NONE
gpiochip_irqchip_add_key()
irq_domain_add_simple()
gpiochip_set_chained_irqchip()
&foo_gpio_irq_chip,
parent_irq,
foo_gpio_irq_handler()
gpiochip_set_cascaded_irqchip()
irq_set_chained_handler_and_data()
__irq_do_set_handler()
루프: gpio 핀 수 만큼
irq_set_parent()

gpiochip_irqchip_add()

include/linux/gpio/driver.h

```c
static inline int gpiochip_irqchip_add(struct gpio_chip *gpiochip,
                                       struct irq_chip *irqchip,
                                       unsigned int first_irq,
                                       irq_flow_handler_t handler,
                                       unsigned int type)
{
        return gpiochip_irqchip_add_key(gpiochip, irqchip, first_irq,
                                        handler, type, false, NULL);
}
```

Add irqchip to gpiochip to control irqs used by gpio.

gpiochip_irqchip_add_key()

Add irqchip to gpiochip to control irqs used by gpio.

- The handler function passed as the fourth argument often uses a predefined irq core function. Most often, one of the four below is used.
  - handle_edge_irq()
  - handle_level_irq()
  - handle_simple_irq() – common case
  - handle_bad_irq()
- The type of irqchip is set to the type passed as the fifth argument.
  - Most of the time, the value IRQ_TYPE_NONE is used, and in this case, no value is set in the HW.
- The sixth argument is true if called from the handle_nested_irq() function.
- The seventh argument is passed only when lockdep is used, otherwise it receives null.

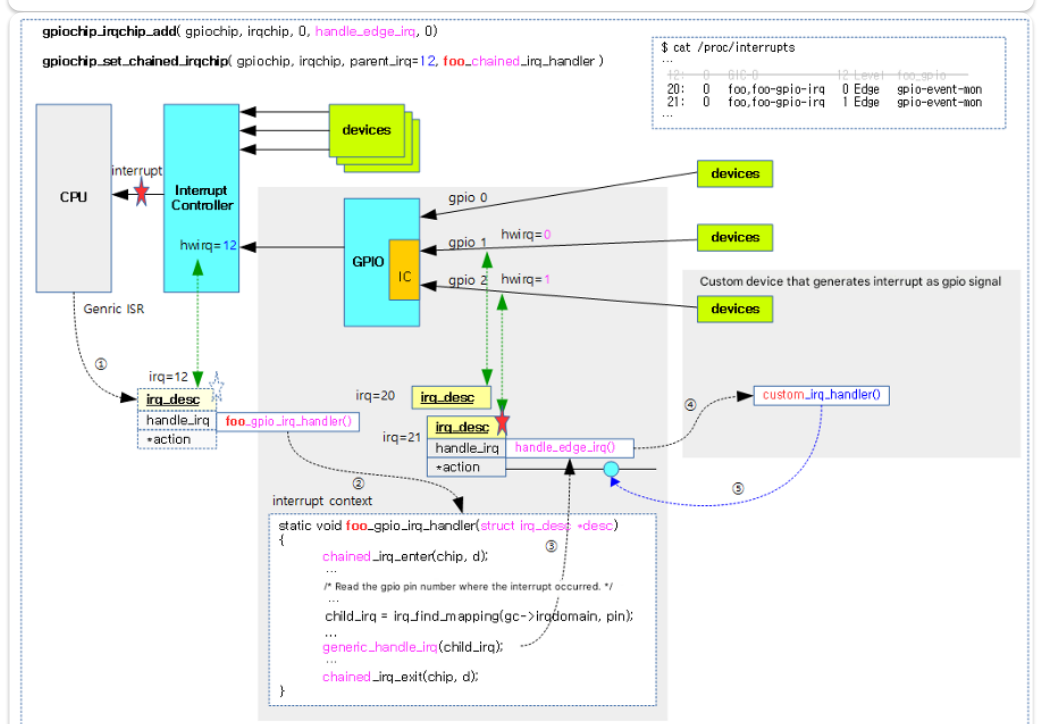
gpiochip_set_chained_irqchip()

This shows how interrupts are processed when they occur when an IRQ and IRQ handler are registered for a GPIO controller capable of chained interrupt processing.

- First, find the descriptor for interrupt 400 that occurred through the ISR of the interrupt controller.
- Calls the handler function foo_gpio_irq_handler() registered in descriptor 400.
- The foo_gpio_irq_handler() function examines the gpio pins that have interrupts set, finds the gpio pin that caused the interrupt, and finds the corresponding child irq number.
- Calls handle_simple_irq(), a generic function provided by the irq subsystem core in the irq descriptor corresponding to the found child irq.
- The handle_simple_irq() function finds and calls the interrupt handler routine of the custom device.
  - Registering an interrupt handler for a custom device uses the following functions.
    - request_irq() & devm_request_irq()
    - request_threaded_irq() & devm_request_threaded_irq()
    - request_any_context_irq() & devm_request_any_context_irq()
    - request_percpu_irq()

The figure below shows how cascaded IRQ processing is handled in chained IRQ mode.

- You can see that the handlers connected to irq12 and irq21 are processed in the irq context.


gpiochip_irqchip_add(gpiochip, irqchip, 0, handle_edge_irq, 0)
gpiochip_set_chained_irqchip(gpiochip, irqchip, parent_irq=12, foo_chained_irq_handler)
CPU
interrupt
Interrupt Controller
hwirq=12
devices
GPIO
IC
gpio 0
gpio 1
gpio 2
hwirq=0
hwirq=1
devices
devices
Custom device that generates interrupt as gpio signal
devices
Genric ISR
①
irq=12
irq_desc
handle_irq
*action
foo_gpio_irq_handler()
irq=20
irq_desc
irq=21
irq_desc
handle_irq
*action
handle_edge_irq()
④
custom_irq_handler()
⑤
②
interrupt context
③

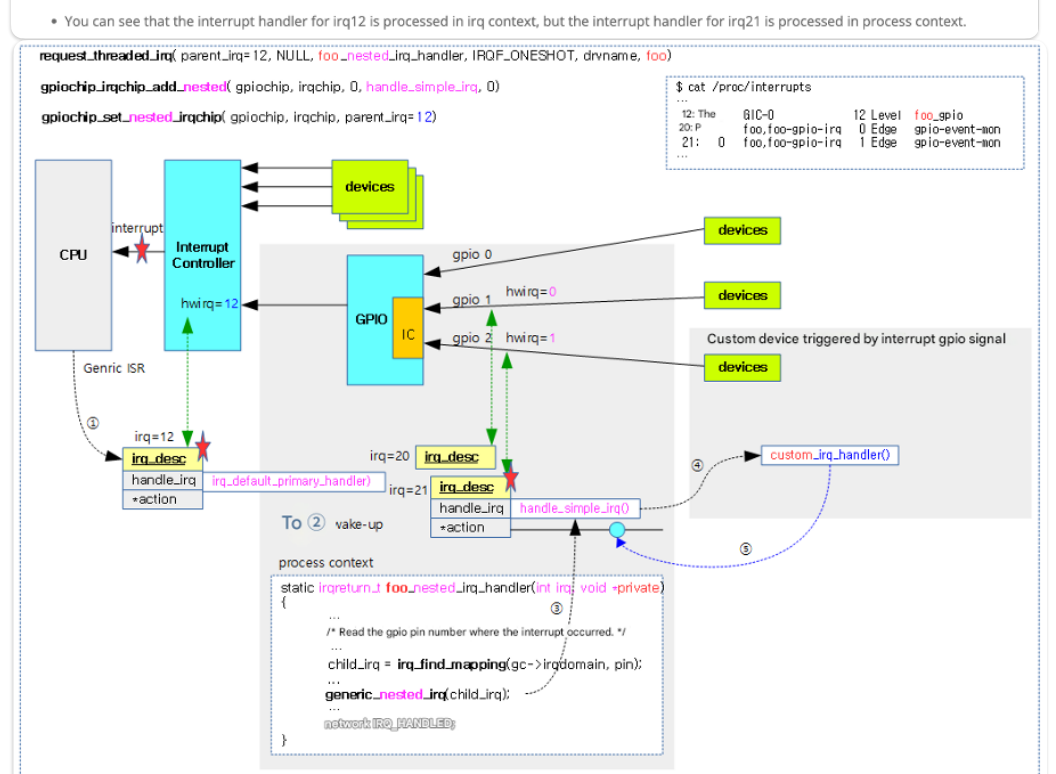
```
$ cat /proc/interrupts
...
~~12:     0   GIC-0              12 Level   foo_gpio~~
20:     0   foo,foo-gpio-irq    0 Edge    gpio-event-mon
21:     0   foo,foo-gpio-irq    1 Edge    gpio-event-mon
...
```

```
static void foo_gpio_irq_handler(struct irq_desc *desc)
{
        chained_irq_enter(chip, d);
        ...
        /* Read the gpio pin number where the interrupt occurred. */
        ...
        child_irq = irq_find_mapping(gc->irqdomain, pin);
        ...
        generic_handle_irq(child_irq);
        ...
        chained_irq_exit(chip, d);
}
```

gpiochip_set_nested_irqchip()

The figure below shows how cascaded interrupt processing is handled using the nested IRQ method.

- You can see that the interrupt handler for irq12 is processed in irq context, but the interrupt handler for irq21 is processed in process context.


request_threaded_irq(parent_irq=12, NULL, foo_nested_irq_handler, IRQF_ONESHOT, drvname, foo)
gpiochip_irqchip_add_nested(gpiochip, irqchip, 0, handle_simple_irq, 0)
gpiochip_set_nested_irqchip(gpiochip, irqchip, parent_irq=12)
CPU
interrupt
Interrupt Controller
hwirq=12
devices
GPIO
IC
gpio 0
gpio 1
gpio 2
hwirq=0
hwirq=1
devices
devices
Custom device triggered by interrupt gpio signal
devices
Genric ISR
①
irq=12
irq_desc
handle_irq
*action
irq_default_primary_handler()
irq=20
irq_desc
irq=21
irq_desc
handle_irq
*action
handle_simple_irq()
To ② vake-up
④
custom_irq_handler()
⑤
process context
③

```
$ cat /proc/interrupts
...
 12: The    GIC-0              12 Level   foo_gpio
 20: P      foo,foo-gpio-irq    0 Edge    gpio-event-mon
 21:    0   foo,foo-gpio-irq    1 Edge    gpio-event-mon
...
```

```
static irqreturn_t foo_nested_irq_handler(int irq, void *private)
{
        ...
        /* Read the gpio pin number where the interrupt occurred. */
        ...
        child_irq = irq_find_mapping(gc->irqdomain, pin);
        ...
        generic_nested_irq(child_irq);
        ...
        return IRQ_HANDLED;
}
```